

SYSTEM DESIGN

Q1. What is System Design?

System design is the process of planning and defining how a software system should be built so that it meets requirements such as **performance, scalability, reliability, security, and maintainability**.

Types of System Design

1. **High-Level Design (HLD)**
 - Focuses on the overall architecture.
 - Defines major components and how they communicate.
2. **Low-Level Design (LLD)**
 - Focuses on implementation details.
 - Defines classes, APIs, database schemas, and algorithms.

Q2. Explain SOLID principles.

SOLID Principles

S — Single Responsibility Principle (SRP)

Every class should have **only one reason to change**.

✗ Bad

```
class Employee{  
  
    void calculateSalary(){}  
  
    void saveEmployee(){}  
  
    void sendEmail(){}  
}
```

One class doing everything.

✓ Good

```
class Employee{}  
  
class SalaryService{}  
  
class EmployeeRepository{}  
  
class EmailService{}  

```

Each has one responsibility.

Interview Question

Why SRP?

Answer

- Easier maintenance
- Easier testing
- Less coupling
- Better readability

O — Open Closed Principle

Open for extension.

Closed for modification.

Bad

```
if(type.equals("Credit"))
```

Every new payment changes code.

Good

```
interface Payment{  
    pay();  
}
```

Implement

CreditCard

UPI

NetBanking

Wallet

No existing code changes.

L — Liskov Substitution Principle

Child class should replace parent without breaking behavior.

Bad

Bird

↓

Penguin

Bird

fly()

Penguin cannot fly.

Violation.

Better

Bird

↓

FlyingBird

↓

Sparrow

I — Interface Segregation

Don't force implementation of unnecessary methods.

Bad

Worker

work()

eat()

Robot must implement eat()

Bad design.

Good

Workable

Eatable

D — Dependency Inversion

Depend on abstraction.

Not implementation.

Bad

```
class Notification{  
  
    EmailService email=new EmailService();  
}
```

Good

```
interface MessageService{}
```

EmailService

SMSService

WhatsappService

Constructor Injection

Q3. What is SOLID?

A set of five object-oriented design principles that improve maintainability, flexibility, scalability, and testability of software.

Q4. Which SOLID principle is used the most in Spring Boot?

Dependency Inversion Principle (DIP), because Spring's dependency injection relies on programming to interfaces rather than concrete implementations.

Q5. Which design patterns are closely related to SOLID?

- **Strategy Pattern** → OCP, DIP
- **Factory Pattern** → OCP, DIP
- **Decorator Pattern** → OCP
- **Observer Pattern** → OCP
- **Dependency Injection** → DIP

Q6. Why is SOLID important?

It helps reduce coupling, increase cohesion, simplify testing, make code easier to extend, and improve long-term maintainability.

Q7. What is a Design Pattern?

A **Design Pattern** is a proven, reusable solution to a commonly occurring software design problem. It is not ready-made code but a template or blueprint for solving problems.

Q8. What are types of Design Pattern?

1. Singleton Pattern –

Singleton ensures only one instance of a class exists and provides a global access point to it. Spring beans are singleton-scoped by default.

Example:

- Database Connection
- Logger
- Configuration Manager
- Cache Manager

Java Example

```
public class DatabaseConnection {  
  
    private static DatabaseConnection instance;  
  
    private DatabaseConnection() {}  
  
    public static DatabaseConnection getInstance() {  
  
        if(instance == null){  
            instance = new DatabaseConnection();  
        }  
  
        return instance;  
    }  
}  
➔ DatabaseConnection db = DatabaseConnection.getInstance();
```

Spring Boot Example

```
@Service  
public class EmployeeService {  
  
}
```

Spring creates only **one bean** by default.

Question: Why Singleton?

It saves memory, maintains a single shared state, and avoids creating unnecessary objects. Spring beans are singleton-scoped by default.

2. Factory Pattern –

Instead of creating objects directly using new, let a factory create them. Factory Pattern centralizes object creation and hides implementation details. Spring's BeanFactory and ApplicationContext are common examples.

Java Example

```
interface Car{

    void drive();

}

class BMW implements Car{

    public void drive(){
        System.out.println("BMW");
    }

}

class Audi implements Car{

    public void drive(){
        System.out.println("Audi");
    }

}
```

Factory

```
class CarFactory{

    public static Car getCar(String type){

        if(type.equals("BMW"))
            return new BMW();

        return new Audi();
    }

}
```

Usage

```
Car car = CarFactory.getCar("BMW");
```

Spring Boot Example

```
ApplicationContext.getBean()
```

Spring creates beans using Factory Pattern.

3. Builder Pattern –

Builder Pattern is useful for creating complex objects with many optional fields. It improves readability and avoids constructor overloading.

Builder Solution

```
Employee emp = Employee.builder()
.name("Rahul")
.age(25)
.salary(50000)
.build();
```

Spring Boot Example

Using Lombok

```
@Builder
public class Employee {

}
```

4. Strategy Pattern –

Strategy Pattern encapsulates multiple algorithms and allows selecting one at runtime without modifying existing code. Strategy Pattern eliminates multiple if-else conditions and follows the Open/Closed Principle.

Spring Boot Example

Different payment services implementing a common interface.

5. Observer Pattern –

Observer Pattern establishes a one-to-many dependency. When one object changes state, all registered observers are notified automatically.

6. Adapter Pattern –

Adapter Pattern allows incompatible interfaces to work together by acting as a bridge between them.

Java Example

```
interface Printer{

    void print();

}
```

Old printer

```
class LegacyPrinter{

    void printOld(){

    }

}
```

Adapter

```
class PrinterAdapter implements Printer{

    LegacyPrinter printer = new LegacyPrinter();

    public void print(){

        printer.printOld();

    }

}
```

Spring Example

Integrating third-party libraries with different interfaces.

7. Decorator Pattern –

Decorator Pattern adds new functionality dynamically without changing the original class.

Spring Boot Example

Adding logging, security, or caching around services without modifying their core logic.

Q9. Which Design Patterns are used in Spring Boot?

- **Singleton** → Spring beans are singleton by default.
- **Factory** → `BeanFactory` and `ApplicationContext` create and manage beans.
- **Builder** → Frequently used with Lombok's `@Builder` and object construction.
- **Strategy** → Multiple implementations of an interface (e.g., payment or notification services).

Q10. List all response codes and their meanings.

1. 200 – OK
2. 201 – CREATED
3. 204 – NO CONTENT
4. 301 – PERMANENT REDIRECT
5. 302 – TEMPORARY REDIRECT
6. 400 – BAD REQUEST
7. 401 – UNAUTHARISED REQUEST
8. 402 – FORBIDDEN
9. 404 – PAGE NOT FOUND
10. 500 – INTERNAL SERVER ERROR